

Laporan Praktikum Kontrol Cerdas

Nama : Widhi Widya Hastuti
NIM : 224308025
Kelas : TKA 6A
Akun Github (Tautan) : <https://github.com/widhiwidya>
Student Lab : Yulia Brilianty
Assistant

1. Judul Percobaan

Reinforcement Learning for Autonomous Control

2. Tujuan Percobaan

- Memahami konsep dasar Reinforcement Learning (RL) dalam sistem kendali.
- Mengimplementasikan agen RL menggunakan algoritma Deep Q-Network (DQN).
- Menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan RL.
- Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
- Menggunakan GitHub untuk version control dan dokumentasi praktikum.

3. Landasan Teori

Reinforcement Learning (RL) adalah metode pembelajaran di mana agen belajar mengambil keputusan melalui interaksi dengan lingkungan. Agen memperoleh reward atau penalty sebagai umpan balik dari tindakannya, sehingga ia dapat mengoptimalkan strategi untuk mencapai tujuan tertentu.. Dalam konteks sistem kendali, RL digunakan untuk mengoptimalkan pengendalian suatu sistem tanpa model eksplisit, memungkinkan agen untuk belajar dari pengalaman langsung melalui interaksi dengan lingkungan (Norman, n.d.). Namun, RL juga memiliki tantangan, seperti waktu pelatihan yang lama, eksplorasi yang dapat menyebabkan hasil buruk di awal, serta kesulitan dalam merancang fungsi reward yang sesuai (Andreanus & Kurniawan, 2018).

Deep Q-Network (DQN) adalah algoritma dalam pembelajaran penguatan (*Reinforcement Learning*) yang menggabungkan *Q-Learning* dengan jaringan saraf dalam (*deep neural networks*) untuk mengatasi masalah dengan ruang keadaan yang besar dan kompleks (Putra et al., 2024). Dengan menggunakan DQN, agen dapat belajar dari lingkungan yang kompleks dengan jumlah kemungkinan kondisi (state) yang sangat besar, sesuatu yang sulit dilakukan jika hanya mengandalkan Q-table seperti pada Q-Learning biasa. Tujuan utama agen adalah untuk menemukan urutan tindakan yang menghasilkan hadiah total terbesar dalam jangka panjang. Untuk itu, DQN menggunakan fungsi nilai Q yang memperkirakan seberapa baik setiap tindakan dalam suatu keadaan tertentu. Selain CartPole, DQN juga dapat digunakan pada lingkungan lain seperti LunarLander-v2, di mana agen belajar mengendalikan pendaratan pesawat luar angkasa, atau MountainCar-v0, di mana agen harus menemukan cara untuk mencapai puncak bukit dengan menggunakan momentum (Gou & Liu, 2019).

LunarLander-v2 adalah environment OpenAI Gym untuk menguji algoritma RL, di mana agen mengendalikan pendaratan wahana luar angkasa di permukaan bulan. Reward diberikan berdasarkan kualitas pendaratan dan efisiensi penggunaan bahan bakar, dengan penalti untuk pendaratan buruk atau keluar dari area. Sementara, MountainCar-v0 adalah environment di mana agen mengayunkan mobil untuk mendapatkan momentum agar mencapai puncak bukit. Agen dapat bergerak ke kiri, kanan, atau diam, dengan reward negatif setiap langkah, mendorong penyelesaian secepat mungkin sebelum batas langkah tercapai.

4. Analisis dan Diskusi

Analisis

Pada minggu keempat Mata Kuliah Praktikum Kontrol Cerdas, kami melakukan percobaan dengan mengimplementasikan Deep Q-Network (DQN) untuk menyelesaikan tantangan MountainCar-v0 menggunakan reinforcement learning. Algoritma ini melatih agen agar dapat mencapai puncak bukit dengan memanfaatkan pengalaman yang diperoleh selama permainan.

Proses dimulai dengan mendefinisikan kelas DQNAgent, yang bertanggung jawab atas logika pembelajaran. Agen ini memiliki memory buffer (deque) yang berfungsi untuk menyimpan pengalaman (experience replay), sehingga pembelajaran tidak hanya bergantung pada pengalaman terbaru tetapi juga pengalaman sebelumnya. Pengambilan

keputusan oleh agen didasarkan pada eksplorasi dan eksploitasi, yang dikendalikan oleh parameter epsilon. Pada tahap awal, agen lebih sering memilih aksi secara acak (eksplorasi), tetapi seiring waktu nilai epsilon berkurang (epsilon_decay), sehingga agen mulai lebih banyak menggunakan pengalaman sebelumnya dalam pengambilan keputusan (eksploitasi).

Dalam bagian utama kode, lingkungan MountainCar-v0 dari Gymnasium diinisialisasi dalam mode human render agar pergerakan mobil dapat divisualisasikan. Agen kemudian dilatih selama sejumlah episode yang telah ditentukan. Pada setiap episode, agen mengamati kondisi lingkungan (state), memilih aksi (act()), menerima umpan balik berupa reward, dan menyimpan pengalaman tersebut ke dalam memory buffer. Jika agen berhasil mencapai tujuan (terminated), ia memperoleh reward yang lebih besar, sedangkan jika masih dalam perjalanan, ia dikenakan penalti kecil untuk mendorong eksplorasi lebih lanjut.

Selain itu, kode ini terintegrasi dengan Pygame untuk mendeteksi apakah pengguna menutup jendela permainan. Jika jendela ditutup, program akan berhenti dan lingkungan akan ditutup menggunakan env.close(). Secara keseluruhan, kode ini menggambarkan proses pembelajaran agen dengan DQN, di mana agen belajar dari pengalaman untuk mengoptimalkan pergerakan dalam lingkungan MountainCar-v0.

Diskusi

Program DQN ini melatih agen untuk menyelesaikan MountainCar-v0 dengan menggunakan experience replay, yang membantu agen belajar lebih stabil dengan mengandalkan pengalaman sebelumnya. Agen mengambil keputusan dengan menyeimbangkan eksplorasi (mencoba tindakan baru) dan eksploitasi (menggunakan pengalaman sebelumnya), yang dikendalikan oleh nilai epsilon. Nilai ini akan berkurang seiring waktu, sehingga agen dapat menemukan strategi terbaik tanpa langsung terjebak dalam pola tertentu.

Sistem reward dirancang agar agen lebih termotivasi mencapai tujuan, meskipun bisa ditingkatkan dengan memberi hadiah tambahan saat mendekati puncak bukit. Dalam pelatihan, penggunaan sampel acak membantu menjaga kestabilan pembelajaran, tetapi ada risiko overfitting. Hal ini bisa diatasi dengan metode prioritized experience replay, yang memungkinkan agen belajar dari pengalaman yang lebih penting.

Namun, karena program menggunakan tampilan visual (human render), prosesnya bisa menjadi lebih lambat, terutama jika dijalankan dalam waktu lama atau pada perangkat

dengan spesifikasi rendah. Salah satu solusinya adalah menjalankan pelatihan tanpa tampilan visual dan hanya mengaktifkannya saat evaluasi. Secara keseluruhan, program ini sudah cukup baik dalam menerapkan DQN, meskipun masih ada peluang untuk meningkatkan eksplorasi, sistem reward, dan efisiensi pembelajaran.

5. Assignment

Program ini dirancang untuk menguji agen yang telah dilatih menggunakan algoritma Deep Q-Network (DQN) dalam lingkungan MountainCar-v0 dari Gym. Tujuan utama agen adalah menggerakkan mobil hingga mencapai puncak bukit dengan cara belajar dari pengalaman sebelumnya.

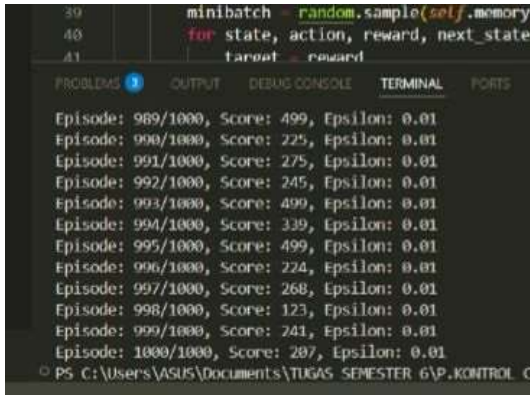
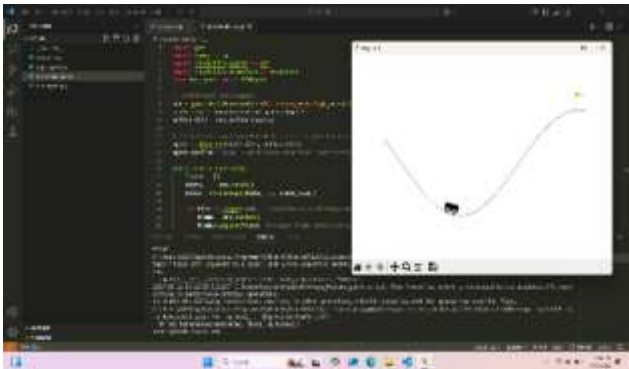
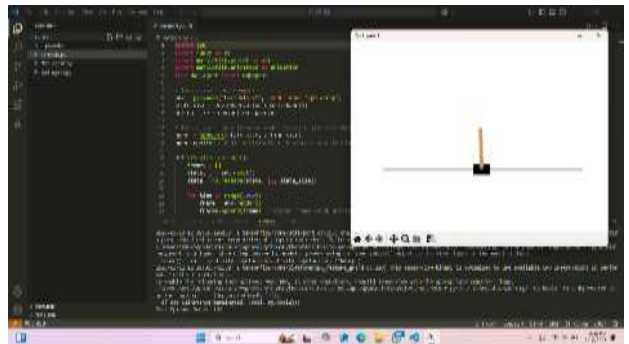
Pertama, program menginisialisasi lingkungan MountainCar-v0 dalam mode `rgb_array`, yang memungkinkan perekaman tampilan dalam bentuk gambar. Kemudian, ukuran state dan jumlah aksi diambil dari lingkungan tersebut. Setelah itu, agen DQN dibuat menggunakan kelas `DQNAgent`. Untuk pengujian, nilai epsilon disetel rendah (0.01) agar agen lebih mengandalkan pengalaman yang sudah dipelajari daripada melakukan eksplorasi secara acak.

Selanjutnya, fungsi `visualize_episode()` digunakan untuk menjalankan satu episode simulasi. Dalam proses ini, setiap frame dari lingkungan disimpan ke dalam daftar frames untuk keperluan animasi. Agen memilih aksi menggunakan metode `act()`, lalu lingkungan diperbarui berdasarkan aksi tersebut. Jika mobil berhasil mencapai posisi 0.5 (puncak bukit dengan bendera), agen diberikan reward tambahan sebesar 10 untuk mempercepat pembelajaran, dan episode langsung dihentikan. Jika episode berakhir karena agen mencapai tujuan atau langkah maksimal tercapai, skor episode ditampilkan di terminal.

Setelah simulasi selesai, program menggunakan fungsi `create_animation(frames)` untuk menampilkan animasi pergerakan agen. Animasi ini dibuat dengan `matplotlib.animation`, di mana setiap frame ditampilkan secara berurutan untuk menghasilkan efek visual seperti video.

Terakhir, setelah animasi selesai ditampilkan, program menutup lingkungan dengan `env.close()` untuk menghemat sumber daya. Secara keseluruhan, program ini menguji performa agen dalam menyelesaikan MountainCar-v0 dengan minim eksplorasi, menampilkan pergerakan agen, serta menerapkan strategi reward tambahan agar agen dapat belajar lebih cepat mencapai tujuan.

6. Data dan Output Hasil Pengamatan

No.	Variabel	Hasil Pengamatan
2.	Data hasil uji enviroentment	 <pre> 39 minibatch = random.sample(self.memory 40 for state, action, reward, next_state 41 target = reward </pre> Episode: 989/1000, Score: 499, Epsilon: 0.01 Episode: 990/1000, Score: 225, Epsilon: 0.01 Episode: 991/1000, Score: 275, Epsilon: 0.01 Episode: 992/1000, Score: 245, Epsilon: 0.01 Episode: 993/1000, Score: 499, Epsilon: 0.01 Episode: 994/1000, Score: 339, Epsilon: 0.01 Episode: 995/1000, Score: 499, Epsilon: 0.01 Episode: 996/1000, Score: 224, Epsilon: 0.01 Episode: 997/1000, Score: 268, Epsilon: 0.01 Episode: 998/1000, Score: 123, Epsilon: 0.01 Episode: 999/1000, Score: 241, Epsilon: 0.01 Episode: 1000/1000, Score: 287, Epsilon: 0.01
3.	Hasil enviroentment Cartpole	
5.	Hasil enviroentment MountainCar	

7. Kesimpulan

- Metode *prioritized experience replay* dapat digunakan untuk membuat agen lebih cepat belajar dari pengalaman yang lebih berpengaruh terhadap keberhasilannya.
- Mode tampilan (*human render*) dalam simulasi memperlambat proses pelatihan. Oleh karena itu, saat pelatihan, sebaiknya tampilan visual dinonaktifkan dan hanya digunakan untuk evaluasi hasil.

- Lingkungan dengan *reward* yang jarang seperti *MountainCar*, membutuhkan eksplorasi yang lebih lama dan *replay buffer* yang lebih besar agar agen dapat mengumpulkan pengalaman yang cukup untuk menemukan strategi yang efektif.
- Dengan DQN, agen mampu belajar bahwa hanya maju terus tidak cukup untuk mencapai tujuan. Agen memahami bahwa terkadang perlu bergerak mundur terlebih dahulu untuk mengumpulkan momentum sebelum mencapai puncak.

8. Saran

Untuk membuat pembelajaran agen dalam menyelesaikan masalah MountainCar-v0 lebih efektif, ada beberapa aspek yang dapat dioptimalkan. Salah satunya adalah dengan menyesuaikan parameter seperti gamma, epsilon decay, dan ukuran memory buffer, sehingga agen bisa belajar lebih efisien dan menemukan strategi optimal dengan lebih cepat.

Selain itu, metode prioritized experience replay dapat digunakan agar agen lebih sering belajar dari pengalaman yang lebih relevan terhadap keberhasilannya. Dengan cara ini, proses pelatihan menjadi lebih fokus dan efektif.

10. Daftar Pustaka

Reinforcement Learning: Sebuah Tinjauan Pustaka. *Jurnal Telematika*, 12(2), 113–118.
<https://doi.org/10.61769/telematika.v12i2.193>

Gou, S. Z., & Liu, Y. (2019). *DQN with model-based exploration: Efficient learning on environments with sparse rewards* (No. arXiv:1903.09295). arXiv.
<https://doi.org/10.48550/arXiv.1903.09295>

Norman, P. (n.d.). *Training reinforcement learning model with custom OpenAI gym for IIoT scenario*.

Putra, R. A., Syahbana, Y. A., & Ananda. (2024). Implementasi Algoritma Deep Q-Network (DQN) pada Lampu Lalu Lintas Adaptif Berdasarkan Waktu Tunggu dan Arus Kendaraan. *The Indonesian Journal of Computer Science*, 13(5). <https://doi.org/10.33022/ijcs.v13i5.4372>